

HAMi 실험 보고서

측정 기간: 2026년 9월 2일 ~ 10월 7일 (5회)
측정 장비: RTX 4080 SUPER 1장, RTX 3060 1장, RTX 2060 SUPER 1장, RTX A6000 2장 (서버 4대)

1. 개요

1.1 검증한 가정

플랫폼은 물리 GPU 한 장을 여러 사용자에게 나눠 주는 방식을 택했고, 그 전제는 다음 한 문장입니다.

물리 GPU 한 장을 여러 개의 격리된 뭉치로 나누어 각 뭉치에 VRAM 상한을 강제할 수 있고, 그 안에서 실제 딥러닝 작업이 정확하게 실행되며, 한 사용자가 다른 사용자의 작업을 망가뜨릴 수 없다.

이 문장의 네 요소(분할, 강제, 정확성, 작업 격리)와, 실제 운영에 필요한 다섯 가지(뭉치 크기 계산, 다중 사용자 성능 분배, 세대 다른 카드와 약한 호스트, 두 장 카드, 여러 노드 운용)를 실험으로 확인했습니다. 분할에는 CUDA 호출을 사용자 공간에서 가로채는 HAMi-core를 Kubernetes 없이 Docker 단독으로 사용했습니다.

1.2 결론 요약

항목	결론
분할·강제·정확성·작업 격리	성립. 상한은 실제로 강제되고, 실제 작업 5종이 분할 상태에서 비트 단위로 같은 결과를 냈으며, 어떤 이웃도 다른 사용자의 결과를 바꾸지 못했습니다
뭉치 크기 계산	카드마다 상수가 다릅니다. 상수는 SM 수 × SM당 최대 스레드 로 예측되며 VRAM 용량과 무관합니다. 일괄 규칙(설정 값 - 256)은 큰 카드에서 깨집니다 → 노드 등록 시 보정 필수
다중 사용자	한 장을 9명까지 나눠도 성능이 1/N로 고르게 나뉘고 카드 전체 처리량은 99 % 유지됩니다. 수용 인원을 정하는 것은 메모리입니다
두 장 카드	장치마다 상한과 메모리 차감이 따로 적용되고, 두 장에 나눠 돌리는 분산 학습(DDP, PyTorch DistributedDataParallel)도 결과가 정확합니다 → 두 장 신청 허용 가능
여러 노드	노드를 풀의 한 차원으로 두는 실행기가 이기종 두 노드에서 과할당·회수 누락 없이 동작했습니다
VRAM 용량 격리	성립하지 않는 경로가 있습니다. 사용자 코드 한 줄로 뭉치 바깥의 물리 메모리를 뭉치의 수 배까지 점유할 수 있고, 큰 카드일수록 커집니다(A6000에서 2g 뭉치 약 10GB). 직접 짠 커널의 로컬 메모리처럼 일반 사용 중에도 생길 수 있는 경로가 있을 것으로 보입니다. 운영 전 호스트 쪽 대책이 필요합니다

확인된 결함과 대책은 5절에 모았습니다.

2. 실험 환경과 통제 변수

2.1 장비

항목	RTX 4080 SUPER	RTX 3060	RTX 2060 SUPER	RTX A6000 × 2
아키텍처 / CC	Ada / 8.9	Ampere / 8.6	Turing / 7.5	Ampere / 8.6
SM × SM당 최대 스레드	80 × 1536	28 × 1536	34 × 1024	84 × 1536
VRAM	16GB	12GB	8GB	48GB
CPU / RAM	i7-9700KF 8코어 / 31GB	i7-11700 / 15GB	Pentium G4400 2코어 / 7GB	Xeon Gold 6226R 16코어 / 125GB
비고	단독 서버	디스플레이 연결	약한 호스트	ECC 꺼짐, NVLink 없음, 다른 연구실 소유

2.2 모든 실험에서 통제된 변수

구분	고정한 값	이유
드라이버	580.178.04 (CUDA 13.0), 모든 장비 동일	컨텍스트 크기와 예약분이 드라이버에 따라 달라짐
HAMi-core	커밋 f01e9f23 고정, 상위 저장소 빌드 절차 그대로. 2~5번째 장비의 libvgpu.so 는 비트 단위로 같은 파일(sha256 d75afe70...)	릴리스 태그가 없어 커밋을 고정
컨테이너 런타임	Docker + NVIDIA Container Toolkit 1.20.0, nvidia 런타임 미등록(톨킷 혹은 --gpus)	장비 간 동일 조건
작업 이미지	pytorch/pytorch:2.13.0-cuda13.0-cudnn9-runtime 기반 (PyTorch 2.13.0+cu130, cuDNN 9.20), 장비마다 같은 베이스 digest와 같은 패키지 버전	워밍업 뒤 손실과 초과분은 이미지에 따라 달라짐
컨테이너 사양	--network none, 비루트 uid, 작업 디렉터리 읽기 전용 마운트, libvgpu.so 읽기 전용 마운트, --shm-size 2~4g	외부 영향 차단, 운영 조건 재현
비교 조건	라이브러리 없음(네이티브) / 라이브러리만 주입 / 상한까지 적용, 세 조건을 같은 작업으로 비교	가로채기 비용과 상한 효과를 분리
측정 기준	메모리는 호스트 NVML v2만 사용(컨테이너 안 NVML은 HAMi가 가상화하므로 근거로 쓰지 않음). 0.1초(일부 0.5초) 간격 표본, 프로세스를 cgroup 경로로 컨테이너에 대응	드라이버 예약분이 섞인 v1 값과 가상화된 값을 배제
측정 도구 검증	정확히 2,048 MiB를 잡는 프로브로 표본기가 2,048.0 MiB 차이를 보고하는지 먼저 확인	도구 오차 배제
정확성 판정	학습 종료 시점 모델 파라미터 전체의 SHA-256. 난수 시드, 결정론적 알고리즘, 데이터 로더 워커 시드와 워커 수 고정. 아키텍처가 다른 카드끼리는 비교하지 않고 같은 카드의 단독 실행과 비교	손실값의 근사 일치가 아니라 비트 일치로 판정
할당 단위	사용 가능량은 OOM이 날 때까지 32 MiB 단위로 할당(첫 장비는 256 MiB 단위)	단위에서 오는 양자화 오차 제거

구분	고정한 값	이유
외부 부하	측정 전 대상 GPU에 다른 프로세스가 없고 사용량이 유틸값과 같을 때만 시작. 측정 중 다른 프로세스가 나타나면 즉시 중단(공유 장비)	다른 사용자의 사용량이 섞이는 것을 배제
데이터	모든 데이터와 가중치를 미리 받아 두고 측정 중에는 네트워크 없이 실행. 장비 간 데이터 파일 md5 일치	다운로드 속도와 데이터 차이 배제

2.3 실험별로 바꾼 변수

실험	독립 변수	추가로 고정한 변수	측정값
상한 강제	설정값 2·4·8·16 GiB	카드, 이미지	사용 가능량, 호스트 물리 정점
분할 상태 정확성	작업 5종, 뭉 크기(최소 통과 뭉, 정점 이하)	시드, 배치, epoch	SHA, 처리량
카드별 상수	카드 4종, 워밍업 유무	뭉 크기와 무관함을 2g·4g(·8g·16g)로 확인	컨텍스트 차감량, 손실, 초과분
차감량의 요인	스레드당 스택 한도 1→16 KiB	빈 컨텍스트, 같은 카드	물리 사용량 기울기
다중 사용자	동시 사용자 수 2~10, 작업 혼합	같은 뭉 크기(1,728 MiB), 같은 작업	개별 지연, 합산 처리량, SHA
약한 호스트	사용자 수, 로더 워커 4·1·0	같은 뭉(1,856 MiB), 컨테이너별 RAM 상한, swap 차단	처리량, 작업당 RAM, CPU
작업 격리	이웃의 동작 4종(OOM 반복, 긴 커널, 강제 종료, 라이브러리 없이 전체 점유), 옆 장치 포화	대상 작업과 뭉	대상 작업의 SHA와 지연
두 장 카드	장치별 상한 조합, 채우는 순서, NCCL P2P 사용 여부, HAMI 유무	같은 모델·데이터·합 배치	장치별 사용 가능량, SHA, 처리량
뭉 바깥 점유	스택 한도 4·16·32·64·120 KiB, 모듈 로딩 방식	뭉 2g	호스트 물리 정점, HAMI가 본 사용량
연산 제한	CUDA_DEVICE_SM_LIMIT 100~10 %	작업 종류	상대 처리량
장시간·운용	시간(3·12시간), 노드 연결 끊김, 실행기 교체, 동시 요청 폭주	뭉 크기, 작업	상한, 회계, SHA, 용량 복귀

3. 실험 결과 — 분할, 강제, 정확성, 작업 격리

3.1 상한은 실제로 강제된다

- 설정값을 넘는 할당은 파이썬에서 잡을 수 있는 OOM 예외로 거절되고, 프로세스는 종료되지 않습니다. 예외 뒤 계산을 이어 가고 메모리를 반환한 뒤 다시 할당하는 것까지 정상이었습니다. 75초 동안 1,096회 연달아 상한을 넘겨도 드라이버 오류(Xid)는 0건이었습니다.
- 상한은 **프로세스가 아니라 뭉에** 걸립니다. 한 컨테이너에서 자식 프로세스 셋이 각각 1,536 MiB를 요구하면 셋째가 거절됩니다.
- NCCL 통신 버퍼, 사용자가 컴파일한 CUDA 확장의 드라이버 직접 호출, cudart를 정적 링크한 실행 파일도 모두 상한 안에서 계산되었습니다.

3.2 분할 상태에서도 결과는 비트 단위로 같다

작업	데이터	최소 통과 몫	정확성	처리량 (네이티브 → 주입 → 분할)
ResNet-18	CIFAR-10	1,728 MiB	비트 일치	3,853 → 3,821 → 3,825 img/s
ResNet-50	Imagenette	6,720	비트 일치	330.8 → 331.5 → 330.5 img/s
BERT-base	SST-2	3,712	비트 일치	490.8 → 489.3 → 488.8 ex/s
GPT-2	WikiText-2	6,784	혼란도 일치	19,580 → 19,555 → 19,521 tok/s
Stable Diffusion 1.5	—	4,608	이미지 해시 일치	1.200 → 1.194 → 1.195 img/s

20회 실행이 모두 분할하지 않은 기준과 같았고, 가로채기 비용은 처리량 기준 1 % 미만이었습니다.

3.3 이웃은 결과를 바꾸지 못한다

이웃의 동작	대상 작업 결과	대상 작업 지연
몫 안에서 OOM 반복 (75초간 1,096회)	비트 일치	1.04배
3초짜리 커널 연속 실행	비트 일치	1.95배
실행 중 강제 종료	비트 일치	1.31배
라이브러리 없이 카드 전체 점유	비트 일치	1.04배
같은 서버의 옆 장치를 OOM까지 채우고 유지 (A6000)	비트 일치	1.004배

속도는 느려질 수 있지만 결과는 바뀌지 않았습니다. 강제 종료된 컨테이너의 메모리는 0.22초 만에 반환되었습니다.

4. 실험 결과 — 운영에 필요한 수치

4.1 몫 하나의 비용은 카드마다 다르다

몫 하나에는 세 가지 값이 있습니다. **설정값**(우리가 정하는 상한), **사용 가능량**(사용자가 실제로 쓸 수 있는 양), **물리 사용량**(카드에서 실제로 소모되는 양)입니다. 두 차이는 몫 크기와 무관한 고정 비용이었습니다(2g~16g에서 모두 같음).

항목 (MiB)	2060 SUPER	3060	4080 SUPER	A6000
드라이버 예약	407	387	455	600
풀 (유휴 여유량)	7,784	11,891	15,904	48,538
컨텍스트 차감량 (HAMi가 몫에서 미리 빼는 양)	90	104	250	262
빈 컨텍스트 1개의 실제 물리 크기 (유휴 카드, 프로세스 1개)	93	112	290	271
손실, 워밍업 없음 (설정값 – 사용 가능량)	96	128	—	288
손실, cuBLAS·cuDNN 워밍업 후	160	192	256	352
초과분 (물리 정점 – 설정값)	+14	0	+32	+26

컨텍스트 차감량과 빈 컨텍스트 크기는 **카드를 몇 명으로 나누었는지와 무관한, 컨텍스트 하나당 값**입니다. 다른 프로세스가 없는 유휴 카드에서 프로세스 하나가 빈 컨텍스트 하나를 만들 때 늘어난 장치 사용량(빈 컨텍스트 크기)과, 같은 프로세스가 몫 안에서 본 여유의 감소(차감량)로 잦습니다. 컨텍스트는 GPU를 쓰는 프로세스마다 하나씩 생기므로, 카드를 N명

이 나눠 쓰면 이 비용도 N번 들고, 한 사용자가 프로세스를 둘 띄우면 자기 몫에서 두 번 빠집니다. 3060의 112에는 디스플레이 연결로 인한 8 MiB가 포함되어 있습니다.

손실과 초과분도 몫 크기(2g~16g)와 무관하게 같았으므로, 몫 하나(컨테이너 안 프로세스 하나)당 고정 비용으로 보면 됩니다.

A6000 두 장은 모든 값이 MiB 단위까지 같았습니다. 같은 모델의 카드라면 장치마다 다시 썰 필요는 없습니다.

손실 중 작업 공간 부분은 할당 API를 거치므로 HAMi가 몫 안에서 제한합니다. 반면 초과분은 HAMi가 보지 못하는 컨텍스트 증가분(커널 코드 적재 등)이어서, 위 값은 측정한 작업에서의 값이고 실행하는 커널에 따라 달라질 수 있습니다(5.1절).

4.2 차감량은 스레드 수로 예측되고 VRAM과 무관하다

HAMi-core 소스를 확인한 결과 컨텍스트 차감량은 고정 상수가 아니라, **초기화 시점에 그 카드에서 켜진 컨텍스트의 크기**입니다. 여기서 **스택**은 CPU 프로그램의 호출 스택과 같은 개념으로, GPU 커널의 각 스레드가 함수 호출, 지역 변수, 레지스터에 다 들어가지 않는 값을 두는 스레드 전용 메모리(CUDA 용어로 로컬 메모리)입니다. **스택 한도**는 스레드 하나가 쓸 수 있는 스택 크기의 상한이고 기본값은 1 KiB입니다. 드라이버는 어떤 스레드가 실제로 쓸지 모르므로, 컨텍스트를 만들 때 **카드에 동시에 올라갈 수 있는 모든 스레드(SM 수 × SM당 최대 스레드)** 몫의 스택을 VRAM에 미리 잡아 둡니다. 빈 컨텍스트에서 이 스택 한도를 늘리면 물리 사용량이 정확히 $SM 수 \times SM당 최대 스레드 \div 1024 \text{ MiB/KiB}$ 씩 늘어났습니다(3060 42.0, 2060 SUPER 34.0, A6000 126.0). 이 값을 이 보고서에서는 **스택 기울기**(스택 한도를 1 KiB 늘릴 때 늘어나는 물리 사용량)라고 부릅니다. 컨텍스트에서 가장 큰 가변 성분이 모든 하드웨어 스레드의 스택 예약이라는 뜻입니다.

$$\text{컨텍스트 차감량} \approx 26 \text{ MiB} + 1.87 \text{ KiB} \times (\text{SM 수} \times \text{SM당 최대 스레드})$$

이 식은 세 카드(2060 SUPER 34,816 스레드 → 90 MiB, 3060 43,008 → 104 MiB, 4080 SUPER 122,880 → 250 MiB)의 차감량 실측값을 **카드의 총 스레드 수(SM 수 × SM당 최대 스레드)**를 변수로 한 **선형 회귀**로 맞춘 결과입니다. 기울기 1.87 KiB는 스레드 하나당 드는 비용으로, 그중 1 KiB는 기본 스택 한도(스레드당 1 KiB)로 설명되고 나머지 약 0.87 KiB는 드라이버의 다른 스레드별 예약으로 보입니다. 절편 26 MiB는 스레드 수와 무관한 고정 비용입니다. 두 성분 모두 더 쪼개어 확인하지는 않았습니다.

점이 세 개뿐이므로, 식을 세울 때 쓰지 않은 A6000에서 **측정 전에 기록한 예측**(261.6 MiB)으로 검증했고 실측은 262 MiB(오차 0.1 %)였습니다. A6000은 4080 SUPER와 SM 구성이 거의 같고(84 대 80 SM, 둘 다 SM당 1536) VRAM은 세 배인데, 차감량 차이(12 MiB)가 스레드 차이로 예측한 값(11.2 MiB)과 맞았습니다. **VRAM 용량은 차감량의 요인이 아닙니다.**

워밍업 뒤 손실은 차감량에 cuBLAS·cuDNN 작업 공간 약 70~90 MiB를 더한 값이었습니다.

4.3 일괄 규칙 "설정값 - 256"은 큰 카드에서 깨진다

첫 장비(4080 SUPER)에서 얻은 "사용 가능량 = 설정값 - 256"은 작은 카드에서는 지나치게 보수적이고(2060 SUPER에서 몫마다 96 MiB 낭비, 8명이면 0.75GB), **큰 카드에서는 틀립니다.** A6000의 손실은 352 MiB여서, 이 규칙대로 사용 가능량을 안내하면 몫마다 96 MiB를 더 약속하게 되고 사용자는 안내받은 양을 다 쓰기 전에 OOM을 겪습니다. 측정 전에 예측한 방향(320~350 MiB, 256 초과) 그대로였습니다.

4.4 등급표

인원 N에 대해 $N \times (\text{등급} + 64) \leq \text{풀}$ 을 만족하는 가장 큰 64 MiB 정렬 값입니다. 사용자가 쓸 수 있는 양은 등급에서 4.1절의 워밍업 뒤 손실을 뺀 값입니다.

카드당 인원	2060 SUPER	3060	4080 SUPER	A6000
1	7,680	11,776	15,808	48,448
2	3,776	5,824	7,872	24,192
3	2,496	3,840	5,184	16,064
4	1,856	2,880	3,904	12,032
6	1,216	1,856	2,560	8,000
8	896	1,408	1,920	5,952

4080 SUPER에서는 각 등급을 해당 작업으로 가득 채워 확인했습니다(8인: ResNet-18 실습, 4인: BERT-base 미세 조정, 3인: Stable Diffusion 추론, 2인: ResNet-50·GPT-2). 2의 거듭제곱으로 등급을 잡으면 같은 카드에서 한 자리씩 줄어듭니다(2 GiB면 7명, 1,920 MiB면 8명).

4.5 사용자 수에 따른 성능 분배

동시 사용자	개별 지연 (평균)	카드 전체 처리량	최악/최선 편차	정확성
2	2.00	0.971	1.002	전부 일치
4	3.90	0.978	1.016	전부 일치
6	5.78	0.985	1.028	전부 일치
8	7.67	0.993	1.040	전부 일치
9	8.62	0.991	1.049	전부 일치
10	9명 정상, 10번째는 시작 시 깨끗한 OOM	—	—	9명 일치

- 지연은 인원에 비례하고(오차 4 % 이내), 카드 처리량은 거의 줄지 않았습니다. **한계를 정하는 것은 메모리였고**, 10 번째 사용자는 이미 실행 중인 9명에게 영향을 주지 않고 시작 단계에서 거절되었습니다.
- 8명 동시 시작에서도 초기화 실패가 없었고, 13초 안에 모두 학습에 들어갔습니다.

성격이 다른 작업을 섞을 때. 성격이 다른 작업을 섞으면 GPU 시간이 사용자 수대로 똑같이 나뉘지 않고, **GPU에 일을 쉬지 않고 밀어 넣는 작업이 더 많이 가져갑니다.**

함께 실행한 작업	균등하게 나뉜다면	큰 작업의 지연	작은 작업의 지연
ResNet-50 1개 + ResNet-18 5개	각 6배	3.31배	5.85~5.97배
BERT 1개 + ResNet-18 6개	각 7배	6.39배	6.41~6.65배

ResNet-50은 6분의 1이 아니라 약 3분의 1의 GPU 시간을 가져갔고, 그만큼 ResNet-18들이 손해를 봤습니다.

원인(추정). 연산 제한을 걸지 않으면 HAMI는 연산을 나누지 않고, 여러 컨테이너의 프로세스가 GPU 드라이버의 기본 시분할을 그대로 공유합니다. 드라이버는 프로세스(컨테이너)별 작업 대기열을 번갈아 실행하는데, 차례가 온 프로세스에 대기 중인 커널이 없으면 바로 다음 프로세스로 넘어갑니다. ResNet-50처럼 큰 커널을 연달아 쌓아 두는 작업은 자기 차례를

꼭 채워 쓰는 반면, ResNet-18처럼 작은 커널을 짧게 실행하고 CPU 쪽 커널 실행 요청·동기화·데이터 준비를 기다리는 작업은 차례가 와도 실행할 커널이 없는 순간이 많아 차례를 일찍 넘겨줍니다. 그 빈 시간을 큰 작업이 가져가므로, 결과적으로 GPU 시간이 **각 작업이 GPU를 얼마나 쉬지 않고 점유하려 하는지에 비례해** 나뉩니다. BERT처럼 큰 작업도 동기화가 잦으면 이 쓸림이 거의 없습니다. 드라이버의 시분할 단위와 전환 규칙은 공개되어 있지 않아, 이 설명은 측정 결과에서 추론한 것입니다.

같은 작업끼리 나눠 쓸 때는(4.5절 표) 모두 같은 방식으로 GPU를 쓰므로 1/N로 고르게 나뉘었습니다. 성격이 다른 작업이 섞이면 **긴 커널을 쌓는 작업이 GPU 시간을 더 가져가는 불공정이 생기며**, 연산 제한(4.11절)으로 줄일 수는 있지만 이미 실행된 긴 커널은 끝까지 돌아 완전히 막지는 못합니다.

4.6 약한 호스트

2코어 Pentium과 7GB RAM의 2060 SUPER 서버에서 ResNet-18 1,856 MiB 몫을 1~4명, i7 서버의 3060에서 1~6명 실행했습니다(총 56건).

- 56건 모두 정상, 해시 일치, 호스트 OOM·swap·재시도 0.
- 2코어 호스트는 단독 실행부터 CPU 100 %였지만 처리량(1,794 img/s)은 i7 호스트(1,821 img/s)와 거의 같았습니다. 로더 워커를 0개로 줄여도 같았습니다.
- 작업당 RAM은 약 1.0~1.2GB였고, RAM이 허용하는 인원(5명)보다 **VRAM 등급이 허용하는 인원(4명)이 먼저 한계에** 닿았습니다.
- 반면 4080 SUPER 서버(8코어)에서 8명이 로더 워커 4개씩 쓰면 CPU가 포화해 시작이 느려졌습니다. JPEG 디코딩이 많은 작업은 측정하지 않았습니다.

4.7 두 장 카드

확인 항목	결과
장치별 상한 독립성	장치 0 = 4g, 장치 1 = 8g를 한 컨테이너에 걸고 한쪽을 OOM까지 채워도 다른 쪽 여유는 변하지 않았습니다. 순서를 바꿔도 같았고, 물리 정점은 각 상한 이내였습니다
컨텍스트 차감	각 장치에서 따로 262 MiB씩 차감
상한 지정 방식	<code>CUDA_DEVICE_MEMORY_LIMIT_<i></code> 가 공통값보다 우선합니다. <code>_i</code> 만 주고 공통값을 주지 않으면 지정하지 않은 장치는 상한이 없습니다 . <code>_i</code> 는 호스트 번호가 아니라 컨테이너 안 순번 입니다
장치 간 복사	속도와 체크섬이 네이티브와 같고, 대상 버퍼도 몫의 사용량에 정상 반영, 숨은 메모리 없음
두 장 분산 학습(DDP)	ResNet-18을 두 프로세스(rank)로 나눠 학습, 다섯 구성(네이티브, HAMi, HAMi 반복, P2P 끄 ×2)의 결과 SHA가 모두 같음 . HAMi 처리량 손실 1.3~1.7 %
NCCL 통신 버퍼	한 장에서도 두 장에서도 몫 안에서 계산됨. 4 GiB 몫을 3.5 GiB까지 채운 뒤 초기화하면 정확히 4 GiB에서 거절, 대조군은 5GB까지 증가
통신 경로(A6000 서버)	NCCL이 P2P를 끄지 않아도 공유 메모리(SHM)로 통신. 장치 간 복사 속도도 PCIe 직접 전송 기대치보다 낮음

4.8 장시간 유지와 반복 실행

- **12시간 유지:** 4 GiB 몫을 12시간 유지하며 10분마다 학습을 돌린 결과, 상한(3,456 MiB), 새 프로세스가 본 여유(3,469 MiB), 학습 결과 해시가 끝까지 변하지 않았습니다. 73회 작업 모두 완료, Xid 0, 종료 후 카드 사용량은 시작 전과 같았습니다.

- **3시간 반복 실행:** 실행기로 작업을 3시간 동안 쉬지 않고 시작·종료하며 204건을 처리한 결과, 모두 성공했고, 매 회차 용량이 원래 값으로 돌아왔으며, 남은 프로세스와 새어 나간 용량은 없었습니다.

4.9 여러 노드 운용

노드를 풀의 한 차원으로 두는 실행기로 세대가 다른 두 노드(3060, 2060 SUPER)를 하나의 풀로 운용했습니다. 실행기는 노드에 상주 프로세스나 설치를 남기지 않고, 연결할 때마다 SSH로 에이전트를 보내 실행합니다.

시험	결과
동시 요청 폭주 (합산 풀의 약 3배, 24건)	모두 정상, 풀 초과 0회(50 ms 감시), 물리 사용량이 설정값 + 64를 넘는 경우 0건, 용량 복귀
실행 중 SSH 연결 강제 끊김 (3회)	1~2초 안에 재연결, 이중 계산 없음, 세션 6개 모두 할당을 빠짐없이 수행
실행기 비정상 종료 후 새 실행기 인수	남은 컨테이너 4개를 모두 찾아 차감, 풀 초과 없음
특정 노드에만 들어가는 큰 작업	그 노드를 예약하지 않으면 뒤의 작은 작업들에 밀려 대기 → 노드 예약 규칙으로 해결
원격 제어 지연	중계 서버 경유 요청당 약 8 ms. 시작 지연은 노드의 Docker가 좌우(<code>docker run</code> 중앙값 0.23~0.32초)
시작 경합 재현 시도	같은 노드에 50쌍(컨테이너 100개)을 동시 시작, 재현 0회

4.10 사용자 작업 환경

- 권한 없는 사용자(읽기 전용 이미지, uid 고정, 모든 권한 제거)로도 사용자 영역 패키지 설치, Jupyter Lab, TensorBoard, 뭇 한도까지의 GPU 할당이 모두 동작했습니다.
- 단, **홈을 Docker 기본 임시 파일 시스템에 두면** 실행 권한이 없어 Jupyter(pyzmq)와 TensorBoard(grpc)가 알아보기 어려운 오류로 죽습니다. 실행 가능한 볼륨을 홈으로 주면 정상입니다.
- 컴파일러를 넣은 개발용 이미지에서 무권한 사용자가 CUDA 커널을 컴파일해 22초 만에 실행했고, 상한과 우회 차단이 그대로 유지되었습니다. 이미지는 6.3GB → 21.3GB입니다.

4.11 연산 제한

핵심: 긴 커널을 쓰는 작업은 GPU 시간을 다른 작업보다 더 가져가고, 연산 제한으로도 이를 완전히 막지 못합니다. 연산 제한은 커널을 실행하기 직전에만 개입하므로, 일단 실행된 긴 커널은 끝까지 돌기 때문입니다.

동작 방식. HAMi의 연산 제한(`CUDA_DEVICE_SM_LIMIT`)은 GPU 스케줄러에 개입하지 않고 **커널 실행 요청을 늦추는** 장치입니다. 컨테이너마다 장치별 토큰 통이 있고, 감시 스레드가 그 컨테이너의 실제 GPU 사용률을 보며 설정한 비율에 맞게 토큰을 채웁니다. 프로세스가 커널을 실행하려 할 때 토큰을 쓰고, 토큰이 없으면 실행 요청을 잠시 재웁니다(`nanosleep`). 그 결과 한 작업이 GPU에 넣을 수 있는 일의 양에 **위쪽 상한**이 생깁니다. 8명에게 모두 12.5 %를 걸면 무거운 작업도 자기 상한 이상은 넣지 못해, 4.5절에서 본 쏠림(남이 넘긴 시간을 큰 작업이 가져감)이 줄어듭니다. 보장이 아니라 상한입니다.

측정 결과 (상대 처리량, 제한 없음 = 1.00)

설정	큰 행렬곱 연속	작은 커널 다수	ResNet-18 학습
100 %	1.00	1.00	1.00
75 %	0.97	0.89	—
50 %	0.92	0.96	0.54

설정	큰 행렬곱 연속	작은 커널 다수	ResNet-18 학습
25 %	0.74	0.54	0.25
10 %	0.32	0.21	—

- **긴 커널은 상한을 크게 넘습니다.** 제한이 실행 직전 대기 방식이라 한번 실행된 긴 커널은 끝까지 돕니다. 10 %를 걸어도 큰 행렬곱은 32 %를 썼습니다. 8명 등급(12.5 %)이라면 큰 커널 작업이 상한의 2~3배를 쓸 수 있습니다.
- **불공정을 만드는 작업이 가장 덜 제한됩니다.** 쓸림을 만드는 것은 긴 커널을 쌓는 작업인데, 바로 그 작업이 연산 제한에 가장 덜 반응합니다. ResNet-18처럼 작은 커널 작업은 거의 설정값대로 줄었습니다.
- **남는 시간을 버립니다.** 제한된 작업은 카드가 비어 있어도 대기합니다. 제한 없는 작업과 30 %로 제한한 작업을 함께 돌리면, 제한 없는 쪽이 먼저 끝난 뒤에도 제한된 쪽은 계속 대기했습니다.
- **경합 시 우선순위 수단으로는 동작합니다.** 한쪽에만 제한을 걸면 다른 쪽이 확실히 앞섰습니다. 소유 연구실 작업을 손님 작업보다 앞세우는 용도로 쓸 수 있습니다.
- **섞인 작업에 같은 제한을 걸었을 때의 실제 배분은 측정하지 않았습니다(7절).**

따라서 연산 제한은 **GPU 시간의 쓸림을 줄이는 수단이지 공정한 배분을 보장하는 수단은 아니며**, 이 값을 성능의 백분율로 안내해서도 안 됩니다.

5. 확인된 결함과 대책

5.1 뭇 바깥 물리 점유 (VRAM 용량 격리 실패) — 가장 중요

HAMi는 컨텍스트를 초기화 시점의 크기로 한 번만 뭇에서 빼고, 그 뒤 **컨텍스트 자체가 커지는 것은 보지 못합니다**. 사용자가 제어할 수 있는 두 경로를 2g 뭇에서 확인했습니다. 첫째는 스택 한도(4.2절)를 늘리는 것입니다. 사용자는 `cudaDeviceSetLimit` 한 줄로 스레드당 스택 한도를 바꿀 수 있고, 드라이버는 늘어난 크기 × 카드의 모든 스레드만큼 VRAM을 추가로 잡는데, 이 증가는 할당 API를 거치지 않아 HAMi가 보지 못합니다. 둘째는 커널 코드를 처음부터 모두 적재하는 모듈 로딩 방식입니다. 할당 API로 상한을 넘는 것은 계속 막히지만, 물리 점유는 막히지 않았습니다.

경로	사용자가 할 일	2060 SUPER	A6000
기준	없음	2,028	2,074
스택 한도 4 KiB	<code>cudaDeviceSetLimit(cudaLimitStackSize, 4K)</code> 한 줄	—	2,450 (×1.2)
스택 한도 16 KiB	같음	—	3,962 (×1.9)
스택 한도 32 KiB	같음	—	5,978 (×2.9)
스택 한도 64 KiB	같음	4,168 (×2.0)	10,010 (×4.9)
스택 한도 120 KiB	같음	6,108 (×3.0)	약 17,000 (×8.3, 식으로 계산)
즉시 모듈 로딩	<code>CUDA_MODULE_LOADING=EAGER</code>	2,564 (+516)	3,018 (+944)

(단위: 호스트에서 젠 컨테이너 물리 정점 MiB. 모든 경우 HAMi가 본 뭇 안 사용량은 변하지 않았습니다.)

- 물리 정점은 `설정값 + 초과분 + 스택 기울기 × (스택 한도 - 기본값 1 KiB)` 로 계산되며, A6000의 네 측정값은 이 식과 2 MiB 이내로 맞았습니다. 스택 기울기는 4.2절의 `SM 수 × SM당 최대 스레드 ÷ 1024 MiB/KiB`(A6000 126, 2060 SUPER 34)입니다. 예컨대 A6000에서 64 KiB면 $2,048 + 26 + 126 \times 63 \approx 10,012$ MiB입니다. **스레드가 많은 큰 카드일수록 같은 한 줄의 피해가 커집니다.**

- 물리 메모리가 넘치면 **다음에 들어온 사용자가 하드웨어 OOM으로 실패합니다**. 다른 사용자의 결과가 틀리지는 않으므로 정확성과 작업 격리는 유지되지만, VRAM 용량 격리는 성립하지 않습니다.
- 정상 작업에서도 HAMi가 보지 못하는 증가분이 수십 MiB 있습니다(A6000 약 52 MiB). 측정된 작업에서는 초과분과 여유분 64 MiB 안에 들어갔습니다.

HAMi가 보는 메모리와 보지 못하는 메모리. 라이브러리와 사용자 코드의 GPU 메모리는 두 경로로 잡힙니다.

경로	예	HAMi	결과
할당 API(<code>cuMemAlloc</code> , <code>cuMemCreate</code> 등)	텐서, cuBLAS·cuDNN 작업 공간, NCCL 통신 버퍼	호출마다 계산해 뒀을 넘으면 거절	뒀 안에서 동적으로 제한됨 (손실에 포함)
드라이버가 컨텍스트 안에서 직접 잡는 메모리	커널 코드 적재, 스레드별 로컬 메모리(스택) 예약, printf 버퍼, 디바이스 힙	가로채지 않음	뒀 바깥 물리 점유 (초과분)

뒤쪽 경로의 크기는 **어떤 커널을 실행하느냐에 따라 달라지므로 미리 알 수 없습니다**. 기본값인 지연 로딩에서는 실제로 쓰는 커널만 올라가 일반 PyTorch 작업은 수십 MiB에 그쳤지만, 이는 측정된 작업 범위의 경험치입니다. Triton, FlashAttention, 직접 컴파일한 확장처럼 커널을 많이 올리는 작업은 측정하지 않았습니다.

사용자가 직접 짠 커널 (미측정, 추정). 위 표의 스택 한도 경로는 `cudaDeviceSetLimit` 을 명시적으로 부르지 않아도 생길 수 있습니다. 커널이 스레드마다 큰 지역 배열을 쓰거나 레지스터가 모자라 지역 메모리로 넘치면(spill), 드라이버는 그 커널을 실행할 때 스레드당 로컬 크기 $\times$ 카드의 모든 스레드 슬롯 만큼 예약을 **스스로 늘립니다**. 예컨대 스레드당 16 KiB를 쓰는 커널은 A6000에서 약 2GB(16×126 MiB)를 뒀 바깥에 잡을 것으로 예상됩니다. 이 경우 가로채 API 호출이 없고, **악의가 없는 일반 사용자**(CUDA 실습, 개발용 이미지에서 확장을 컴파일하는 연구자)에게도 생길 수 있습니다. CUDA의 동작 방식에서 추론한 것으로, 2g 뒀에서 지역 배열 크기를 바꿔 가며 확인하는 시험은 아직 하지 않았습니다(7절).

대책

원인이 여러 갈래이고 일부는 가로채 호출이 없으므로, **원인과 무관하게 실제 물리 사용량을 보는 호스트 쪽 장치가 기본 방어선**이고 HAMi-core 수정은 보조 수단입니다.

우선 순위	대책	내용	상태
1	호스트 감시 후 종료	실행기가 컨테이너별 물리 정점이 설정값 + 여유분을 넘으면 종료. 모든 경로에 동작하지만 감시 간격 사이에 늘어나는 양은 막지 못함. 큰 카드는 짧은 시간에 많이 잡으므로 감시 간격(0.1초)과 종료 지연을 측정해 피해 상한을 정해야 함	미검증 (현재는 초과 표시만)
1	실측값 기반 배정	배정 차감을 $\max(\text{설정값} + \text{여유분}, \text{실측 물리 사용량})$ 으로 계산. 이미 넘친 컨테이너가 있는 장치에 새 사용자를 넣지 않아, 새 사용자가 하드웨어 OOM을 겪는 것을 막음. 실행 중 증가는 막지 못함	미검증
2	HAMi-core 수정	<code>cuCtxSetLimit</code> 호출을 가로채 증가분($\Delta \times \text{SM} \times \text{스레드}$)을 뒀의 사용량에 더하고 부족하면 거절. 커널 실행 때 저절로 늘어나는 경우는 막지 못함 (커널 실행을 가로채 커널별 로컬 크기를 조회하는 방법은 가능하지만 모든 실행에 비용이 붙음). 모듈 적재량은 미리 알기 어려움	미검증
3	실행기의 <code>CUDA_MODULE_LOADING=LAZY</code> 강제	EAGER 경로 예방	사용자가 프로세스 안에서 바꿀 수 있어 단독으로는 불충분

우 선 순 위	대책	내용	상태
—	안내	사용자가 커널을 직접 짜는 환경(CUDA 실습, 개발용 이미지)에서는 정상 사용 중에도 뭇을 넘을 수 있음을 안내	제안
—	표기	대책이 검증될 때까지 구현 계획의 "용량 격리 보장"(VRAM 용량 격리)을 "할당 API 기준의 상한 보장"으로 낮춰 표기	제안

풀 여유분을 늘리는 방법은 스택 경로의 증가폭에 사실상 제한이 없어 해결책이 아닙니다.

5.2 뭇 크기 계산의 일괄 규칙이 큰 카드에서 틀림

대책: 노드 등록 시 보정을 필수 단계로 둡니다. 보정 도구는 작성해 두었으며 한 장에 약 30초입니다.

1. 유틸 NVML(총량, 예약, 여유)을 읽어 풀로 삼습니다.
2. 상한을 건 프로브로 컨텍스트 차감량, 워밍업 뒤 사용 가능량, 호스트 물리 정점을 잹니다.
3. 아래 식으로 배정과 등급을 계산하고, **카드 모델 + 드라이버 + 작업 이미지** 단위로 저장합니다. 드라이버나 이미지를 바꾸면 다시 보정합니다.

배정 차감(debit) = 설정값 + 여유분, 여유분 = $\max(64, \text{초과분} + 32 \text{를 } 32 \text{ 단위로 올림})$

사용자에게 표시할 사용 가능량 = 설정값 - (워밍업 뒤 손실)

보정을 생략해야 할 때의 차선택은 4.2절의 예측식에 작업 공간 90 MiB를 더해 "설정값 - (차감량 + 90)"으로 안내하는 것입니다. 네 카드에서 실측과 맞았습니다.

여유분은 보정 작업(행렬곱, 합성곱, 학습 몇 스텝)과 실제 작업 5종에서 **젠 경험치이지 보장이 아닙니다**. 초과분은 실행하는 커널에 따라 달라지므로(5.1절), 여유분을 넘는 경우는 5.1절의 호스트 감시와 실측값 기반 배정이 맞습니다.

사용자에게 표시하는 숫자는 설정값이 아니라 사용 가능량이어야 합니다. 배정량과 실제 사용 가능량이 다르면 혼선이 생깁니다.

5.3 환경변수만으로 주입하면 상한이 우회됨

라이브러리를 LD_PRELOAD 로만 주입하면 사용자가 변수를 지우는 것만으로 상한이 사라졌습니다(4 GiB 뭇에서 6 GiB 할당 성공).

대책: /etc/ld.so.preload 에 읽기 전용으로 주입합니다. 변수를 지워도, 로더를 복사해 써도, 정적 링크 바이너리를 써도 상한이 유지되었습니다(검증됨). 이 파일을 고칠 수 있으면 상한이 무의미하므로 **사용자에게 관리자 권한을 줄 수 없습니다**. 시스템 패키지가 필요하면 관리자에게 요청해 이미지에 반영하는 경로를 마련해야 합니다. 이 파일이 있는 컨테이너는 GPU 라이브러리 없이 프로세스를 시작하지 못하므로 CPU 전용 작업용 이미지는 따로 둡니다.

5.4 비동기 할당자에서 CUDA 그래프 캡처 실패

PyTorch의 cudaMallocAsync 백엔드를 쓰면 HAMi가 스트림 캡처 중 장치 번호를 잘못 계산해 CUDA 그래프 캡처가 실패했습니다.

대책: 이미지에서 할당자 설정을 기본값으로 고정해 사용자가 비동기 백엔드를 켜지 못하게 합니다. 기본 할당자에서는 CUDA 그래프와 컴파일 최적화가 모두 정상이고 상한도 지켜졌습니다(검증됨).

5.5 동시 시작 경합

두 컨테이너가 같은 순간에 시작하고 한쪽이 곧바로 큰 메모리를 잡을 때, 다른 쪽이 이웃의 사용량을 자기 것으로 잘못 읽어 자기 컨텍스트도 못 잡고 종료한 사례가 한 번 있었습니다. 이후 같은 크기 조건을 포함해 50쌍을 동시에 시작했지만 재현되지 않았습니다. 이 환경에서는 컨테이너 안 NVML이 같은 PID 네임스페이스의 프로세스만 보여 주어, 이론상 잘못 고르는 경로가 성립하지 않습니다.

대책: 실행기가 노드별 시작 간격 1초를 두고, 시작 30초 안의 OOM은 1회 재시도합니다. 8명 기준 비용은 약 8초입니다. 효과를 입증한 것은 아니지만 비용이 거의 없는 예방책으로 유지합니다.

5.6 장치별 상한 지정의 함정 (두 장 카드)

HAMi에서 VRAM 상한은 환경 변수로 줍니다. 방법은 두 가지입니다.

- `CUDA_DEVICE_MEMORY_LIMIT=8g` : 컨테이너에 보이는 **모든 장치**에 같은 상한(공통값)
- `CUDA_DEVICE_MEMORY_LIMIT_1=8g` : **1번 장치에만** 상한(장치별 값, 공통값보다 우선)

문제는 장치별 값만 주고 공통값을 주지 않을 때입니다. A6000 두 장이 보이는 컨테이너에 `CUDA_DEVICE_MEMORY_LIMIT_1=8g` 만 주면, 1번 장치는 8 GiB로 제한되지만 **0번 장치는 상한이 전혀 없어** 48 GiB 전체를 쓸 수 있었습니다(4.7절). 이 상황은 컨테이너에 장치가 둘 이상 보일 때 생깁니다. 예를 들어 두 장을 신청한 사용자에게 장치마다 다른 크기의 몫을 주면서 한 장치의 값을 빠뜨리거나, 실행기가 `--gpus all` 로 모든 장치를 넣고 상한으로만 사용 범위를 정하려 하면, 상한을 주지 않은 장치는 그 사용자가 제한 없이 쓸 수 있습니다.

또 `_i` 의 번호는 호스트의 GPU 번호가 아니라 **컨테이너 안에서 보이는 순번**입니다. `--gpus device=1` 로 호스트 1번만 넣으면 컨테이너 안에서는 그 장치가 0번이 되므로 `_0` 으로 지정해야 합니다.

대책: 실행기는 컨테이너에 보이는 **모든 장치에 상한을 명시**합니다(공통값을 함께 주거나 장치마다 `_i`). `_i` 는 컨테이너 안 순번으로 계산합니다. 한 장만 배정하는 컨테이너는 `--gpus device=<n>` 으로 그 장치만 보이게 합니다.

5.7 HAMi 초기화가 장치 0을 건드릴 (미확인)

HAMi를 켜 두 장 분산 학습에서 두 번째 프로세스(rank1)가 자기 장치뿐 아니라 **장치 0에도 최대 210 MiB의 물리 사용량**으로 잡혔고, 실행 런처도 장치 0에 10~64 MiB가 잡혔습니다(네이티브에서는 없음). 소스상 HAMi는 초기화 때 장치 0에서 컨텍스트 크기를 재고 놓으므로 그 흔적으로 보입니다. 일시적인지 지속적인지는 아직 구분하지 못했습니다.

대책(잠정): 장치를 하나만 보이게 한 컨테이너는 그 장치가 안에서 0번이 되므로 영향이 없었습니다. 두 장 이상을 보이게 하는 컨테이너는 장치 0이 가득 찬 상태에서 시작되는지 확인하기 전까지 같은 서버의 장치들을 함께 비워 배정합니다.

5.8 그 밖의 운영 요건

결과	대책
작업이 끝날 무렵 큰 덩어리를 요구하면(평가 배치를 학습 배치보다 크게 잡은 GPT-2) 전체의 94 % 지점에서 실패	평가 배치를 학습 배치보다 크게 잡지 않도록 안내. 실행기가 몫의 90 %·97 %에서 경고를 남기고 남은 시간을 추정

결과	대책
실패 원인이 호스트 메모리 부족, 외부 종료, 몫 부족으로 호스트에서 다르게 관찰됨	세 가지를 구분해 안내. 몫 부족은 "배정받은 GPU 메모리가 부족합니다"로 표시
작은 등급에서는 컨텍스트 하나가 몫에서 250 MiB 이상을 차지해, 8인 등급에서 노트북 두 개를 열면 두 번째가 시작하지 못함	실습 등급을 한 단계 올리거나 세션당 커널을 하나로 제한하고 명시
성격이 다른 작업을 섞으면 긴 커널을 쌓는 작업이 GPU 시간을 더 가져가고, 연산 제한으로도 이미 실행된 긴 커널은 막지 못함(4.5·4.11 절)	성적 평가용 좌석에는 연산 제한을 걸어 실행을 줄이되, 공정한 배분을 보장한다고 안내하지 않음 . 같은 과제는 같은 작업이라 1/N로 나뉘므로 실습 좌석은 과제 단위로 같은 카드에 모으는 배치를 함께 검토. 연구용 세션에는 경합 시 우선순위로만 적용. 이 값을 성능 백분율로 표시하지 않음
컨테이너 안 NVML은 가상화된 값	사용량 집계와 과금은 호스트 쪽 측정으로
홈이 실행 불가 파일 시스템이면 패키지가 동작하지 않음	세션마다 실행 가능한 볼륨을 홈으로
실행기 교체 시 이전 실행기의 종료된 컨테이너가 남음	새 실행기가 남은 컨테이너를 인수해 분류와 삭제까지 처리
네트워크를 끈 컨테이너에서 <code>torchrun --standalone</code> 이 컨테이너 이름을 찾지 못해 멈춤	두 장 작업은 127.0.0.1 고정 주소로 랑데부
노드마다 작업 이미지를 따로 빌드하면 이미지 ID가 달라지고, 버전을 고정하지 않은 패키지가 시점에 따라 바뀜	레지스트리의 같은 digest를 배포하고 Dockerfile의 패키지 버전을 모두 고정
카드 한 장의 수용 인원	$N_{\max} = \min(\text{VRAM 등급 인원}, \text{floor}((\text{유휴 가용 RAM} - 0.5\text{GB}) \div 1.2\text{GB}))$, CPU 전 처리가 많은 작업은 별도 측정

6. 플랫폼 설계에 반영할 사항

- 풀의 단위는 장치.** 상한과 메모리 차감이 장치별로 독립이므로 노드가 아니라 장치마다 풀을 둡니다. 같은 모델이면 보장값을 공유합니다.
- 두 장 신청은 허용 가능.** 한 컨테이너의 두 장 분산 학습이 정확하고 손실이 작습니다. 같은 노드의 두 장치에 각각 몫을 걸고, 5.6·5.7의 조건을 지킵니다. 통신 경로(P2P/SHM)는 서버마다 다를 수 있습니다.
- VRAM 용량 격리는 대책 검증 전까지 보장하지 않는다고 표기.** 특히 A6000처럼 큰 카드를 여러 명이 나눠 쓰는 경우 한 사람이 여러 사람을 실패시킬 수 있습니다.
- 소유자가 따로 있는 장비.** 설치·설정 변경 범위의 사전 협의와 반납 시 되돌리기 절차, 소유 연구실이 GPU를 쓸 때 풀에서 그만큼을 빼는 외부 사용량 처리, 측정·점유 시간대 협의가 필요합니다. 장비가 예고 없이 사라질 수 있으므로 사용자 데이터는 노드 바깥에 둡니다.
- 구형 카드.** CUDA 13은 Maxwell·Pascal·Volta를 지원하지 않아 검증한 이미지가 GTX 10 시리즈에서 실행되지 않습니다. Turing 이전 카드를 받으면 CUDA 12 이미지를 따로 유지해야 하므로, 장비 조사 전에 수용 기준을 정해야 합니다.

7. 미검증 항목

항목	확인할 내용
몫 바깥 점유 대책	호스트 감시 종료의 지연과 그 사이 최대 점유량, 실측값 기반 배정, HAMi-core 패치의 차단 효과와 정상 작업 영향
커널 로컬 메모리 자동 확장	2g 몫에서 스레드당 지역 배열 크기를 바꿔 가며 커널을 실행해, cudaDeviceSetLimit 없이도 몫 바깥 예약이 늘어나는지와 그 크기
작업 종류별 초과분	Triton, FlashAttention, JAX, 직접 컴파일한 확장 등 커널을 많이 올리는 작업에서 여유분 64 MiB가 지켜지는지
HAMi 초기화의 장치 0 접촉	장치 0이 가득 찬 상태에서 두 장 컨테이너의 시작 여부, 남은 메모리의 지속 여부
세 노드 이상의 풀	A6000(장치 두 개)을 포함한 이기종 배치, 중간 서버를 거치는 두 단계 접속, 경로 장애, 외부 사용량 반영
저장소 경로	NAS 마운트(SMB만 가능, NFS 꺼짐), 비루트 권한, 체크포인트, NAS 장애 시 컨테이너 동작
노드 재시작	Docker 데몬 재시작 시 상태 복구(공유 서버라 운영자 승인 필요), 실제 재부팅
장치 간 직접 전송	A6000 서버에서 NCCL이 P2P를 쓰지 않고 복사 속도가 낮은 원인
CPU 부하가 큰 작업	JPEG 디코딩이 많은 작업으로 약한 호스트의 수용 인원
최소 등급 실측	작은 카드의 최소 등급(896 / 1,408 MiB)에 실제로 들어가는 작업
섞인 작업의 연산 배분	ResNet-50 1개 + ResNet-18 5개처럼 성격이 다른 작업에 같은 연산 제한(예: 16 %)을 걸었을 때 GPU 시간이 실제로 얼마나 고르게 나뉘는지

별첨 1. 측정 일정

기간	장비	다룬 주제
9월 2일	RTX 4080 SUPER	상한 강제, 정확성, 작업 격리, 실패 형태, 연산 제한, 3시간 반복 실행
9월 2일	RTX 4080 SUPER	다중 사용자, 동시 시작, 등급표, 무권한 사용자, 단계별 메모리
9월 3일	RTX 4080 SUPER	NCCL 통신 버퍼, 12시간 유지, 개발용 이미지, 시작 경합 발견
9월 23일 ~ 28일	RTX 3060, RTX 2060 SUPER	카드별 상수와 예측식, 약한 호스트, 두 노드 풀 운용, 몫 바깥 점유 발견
10월 6일 ~ 7일	RTX A6000 × 2	예측식 검증, 큰 카드의 몫 바깥 점유, 두 장 카드

모든 측정 기간에 Xid는 한 건도 기록되지 않았습니다.

별첨 2. HAMi-core 환경변수 (소스에서 추출)

변수	의미
CUDA_DEVICE_MEMORY_LIMIT / <code><i></code>	장치별 메모리 상한(g/m/k 접미사 또는 바이트). <code><i></code> 가 우선하고, 둘 다 없으면 상한 없음. <code><i></code> 는 컨테이너 안 순번
CUDA_DEVICE_SM_LIMIT / <code><i></code>	연산 사용률 백분율, 기본 100

변수	의미
CUDA_DEVICE_MEMORY_SHARED_CACHE	회계용 공유 메모리 파일, 기본 /tmp/cudevshr.cache (컨테이너마다 /tmp 가 따로라 몫 단위로 동작)
LIBCUDA_LOG_LEVEL	로그 수준 0-4
GPU_CORE_UTILIZATION_POLICY	연산 제한기 강제·해제
ACTIVE_OOM_KILLER	해석은 되지만 이 커밋에서 호출되지 않음

메모리 제한은 cuMemAlloc_v2, cuMemAllocManaged, cuMemAllocPitch_v2, cuMemAllocAsync, cuMemCreate 를 가로채 추적 사용량과 요청량의 합을 상한과 비교합니다. cuCtxSetLimit 은 가로채지 않습니다(5.1절의 원인).

별첨 3. 검증 작업

작업	데이터	구성	기준 결과
ResNet-18	CIFAR-10	3 epoch, batch 128	acc 0.825, 3,853 img/s (4080 SUPER)
ResNet-50	Imagenette-320	3 epoch, batch 64, 224px	acc 0.499, 331 img/s
BERT-base	GLUE SST-2	1 epoch, batch 32, seq 64	acc 0.906, 491 ex/s
GPT-2	WikiText-2	1 epoch, batch 4, 512 블록	PPL 49.88 → 21.95
Stable Diffusion 1.5	—	512×512, 25 step, fp16	1.20 img/s
ResNet-18 (두 장)	합성 데이터 32×32	합 배치 128, 100 step	고정 시드, rank별 SHA 비교